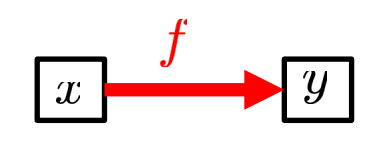
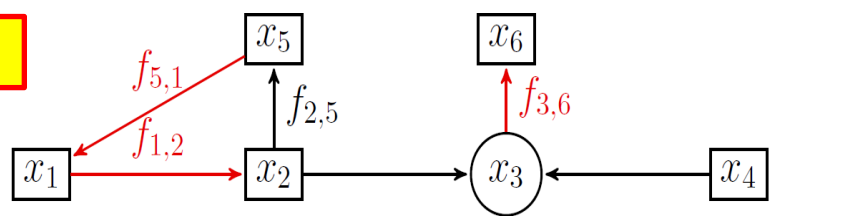


Regression



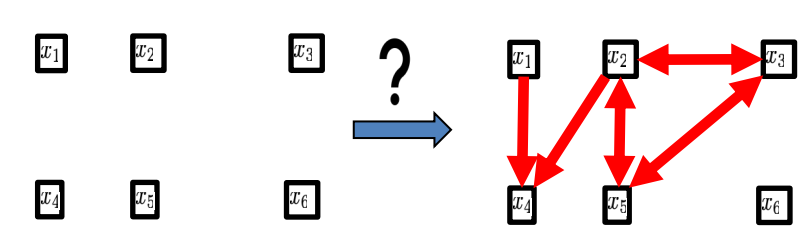
Given $f(X_i) = Y_i$ for $i = 1, \dots, N$, approximate f

Graph Completion



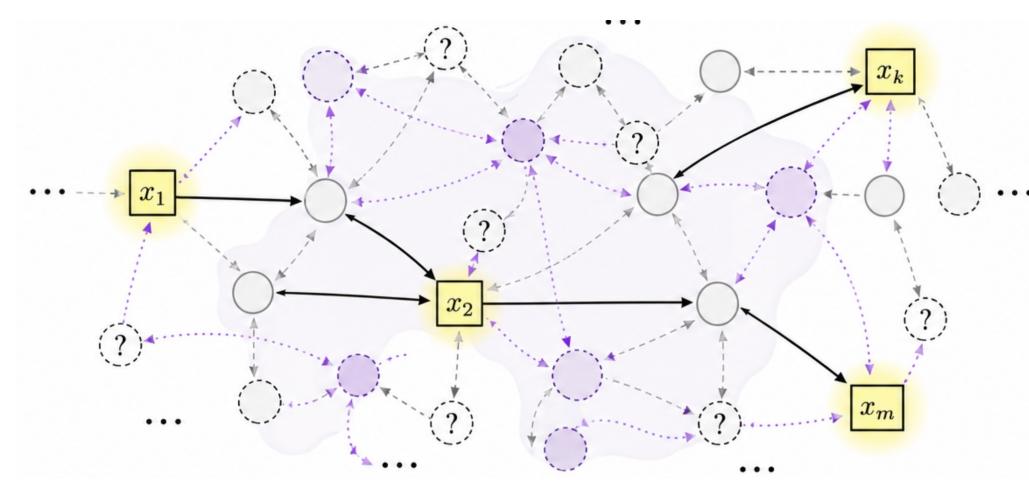
Given N samples from subsets of the variables of the graph, approximate all the unobserved variables and unknown functions.

Graph Discovery



If the graph structure itself is unknown, use samples from the variables of the graph to discover its structure and approximate its unknown functions.

Stochastic graph discovery



Most variables are unobserved and possibly unknown unknowns, and dependencies unknown and possibly stochastic

Discover stochastic dependencies: the smallest subset of observed variables, controls, and memory coordinates needed to reproduce the full stochastic transition law for prediction, UQ, and control.

GP solution for Graph Completion

Replace unknown functions by GPs and compute their MAP/MLE estimator given available data and constraints imposed by the structure of the graph.

$$\begin{cases} \text{minimize} & \|f_{5,1}\|_K^2 + \|f_{1,2}\|_G^2 + \|f_{3,6}\|_F^2 \\ \text{s.t.} & \text{Constraints on data and structure of graph satisfied} \end{cases}$$

Interpretable. Come with simple, transparent and rigorous guarantees. Natural UQ estimates.

Example: Learn following nonlinear PDE

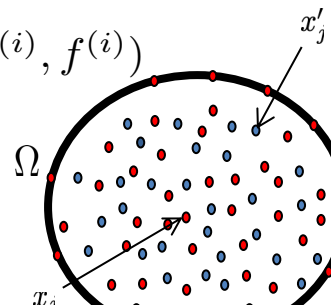
$P(x, u(x), Du(x), D^2u(x)) = f(x)$

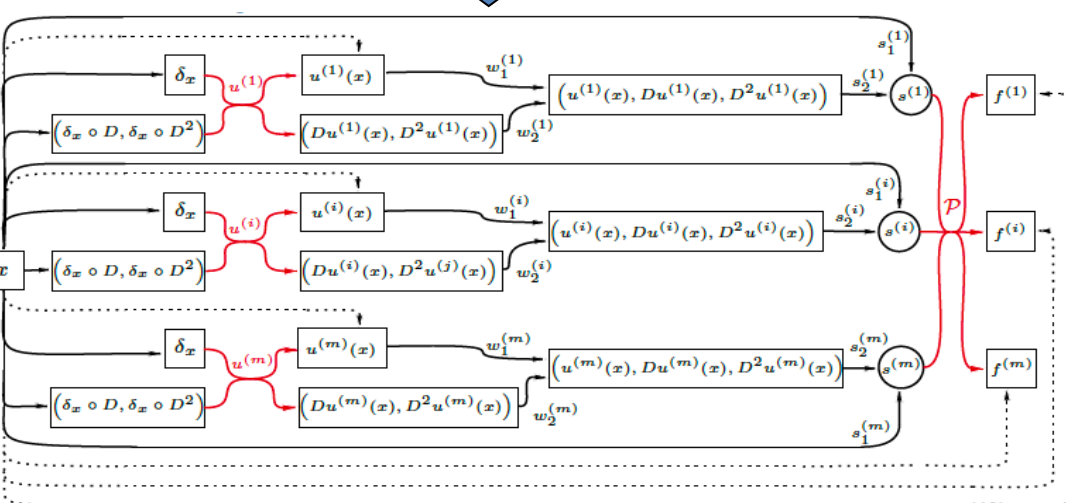
P unknown [Jalalian, Osorio Ramirez, Hsu, Hosseini, O., 2025]

Data: M pairs of solution/source functions $(u^{(i)}, f^{(i)})$ observed at a finite number of points.

$[u^{(i)}(x_1), \dots, u^{(i)}(x_N)],$

$[f^{(i)}(x'_1), \dots, f^{(i)}(x'_{N'})]$





Generalizes to solving PDEs, training a neural-network, generative modeling, system identification, closure modeling, dimension reduction, etc.

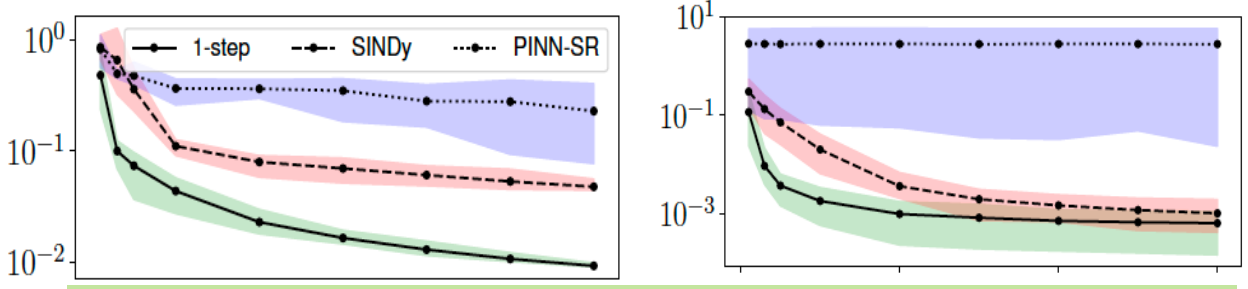
Minimize $\|Q\|_F^2 + \sum_{i=1}^m \|v^{(i)}\|_K^2$

subject to $Q(x'_j, v^{(i)}(x'_j), Dv^{(i)}(x'_j), D^2v^{(i)}(x'_j)) = f^{(i)}(x'_j),$

$v^{(i)}(x_j) = u^{(i)}(x_j)$

$x_j, x'_j \in \Omega, i = 1, \dots, M$

$\|Q_{m,N}\|_F^2 + \sum_{i=1}^m \|v_{m,N}^{(i)}\|_K^2 \leq \|P\|_F^2 + \sum_{i=1}^m \|u^{(i)}\|_K^2$



Dependencies may be invisible in the conditional mean but visible in variance, tails, or multimodality.

GP solution for Graph Discovery

Let $x_1, \dots, x_d$ be d random variables.

Given N samples from $(x_1, \dots, x_d)$

$\forall i$ identify a minimal set $\mathcal{A}(x_i) \subset \{x_1, \dots, x_d\} \setminus \{x_i\}$ such that

$x_i \mid (\{x_1, \dots, x_d\} \setminus \{x_i\}) \stackrel{d}{\approx} x_i \mid \mathcal{A}(x_i)$

and a function T_i such that

$x_i \mid (\{x_1, \dots, x_d\} \setminus \{x_i\}) \stackrel{d}{\approx} T_i(\mathcal{A}(x_i), \omega_i)$

where $\omega_i \sim \mathcal{N}(0, 1)$ is unobserved and independent from $\{x_1, \dots, x_d\} \setminus \{x_i\}$

Example

Assume $i = 1$

Train a conditional generative model $\mathcal{G}_\theta$ for $x_1 \mid \{x_2, \dots, x_d\}$

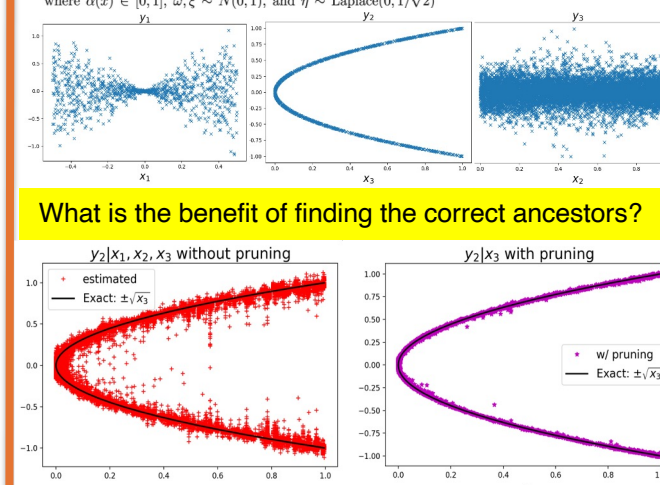
$x_1 \mid \{x_2, \dots, x_d\} \stackrel{d}{\approx} \mathcal{G}_\theta(x_2, \dots, x_d, \omega)$

Use GP-based ANOVA (CHD) to discover functional dependencies between $y := \mathcal{G}_\theta(x_2, \dots, x_d, \omega)$ and $x_2, \dots, x_d, \omega$

Generate N' samples X_i from $X := (x_2, \dots, x_d, \omega)$ and compute the corresponding $Y_i := \mathcal{G}_\theta(X_i)$

Apply CHD to the N' samples (X_i, Y_i)

What is the benefit of finding the correct ancestors?



Increased accuracy and interpretability of the conditional generative model. Increased speed, robustness and cost of the downstream control policy

Example: Multiscale Lorenz-96

$$\frac{dx_j}{dt} = (x_{j+1} - x_{j-2})x_{j-1} - x_j + F - \frac{hc}{b} \sum_{k=1}^J y_{k,j}.$$

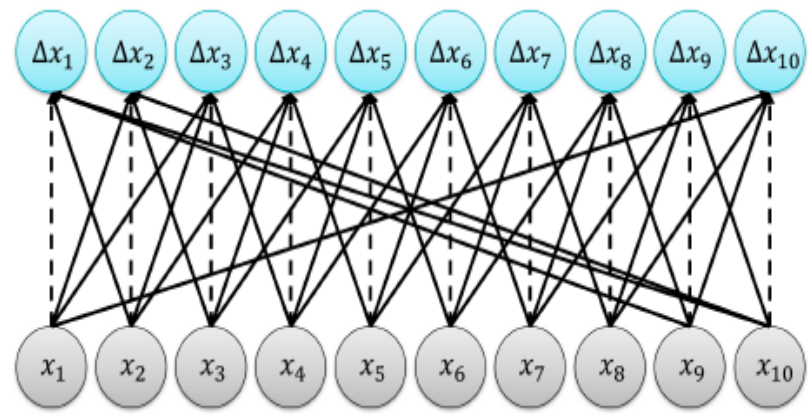
$$\frac{dy_{k,j}}{dt} = -cb y_{k+1,j}(y_{k+2,j} - y_{k-1,j}) - cy_{k,j} + \frac{hc}{b} x_{j,j}.$$

Observed variables: slow state $x = (x_1, \dots, x_m)$.

Hidden variables: fast variables $y_{k,j}, k = 1, \dots, J$.

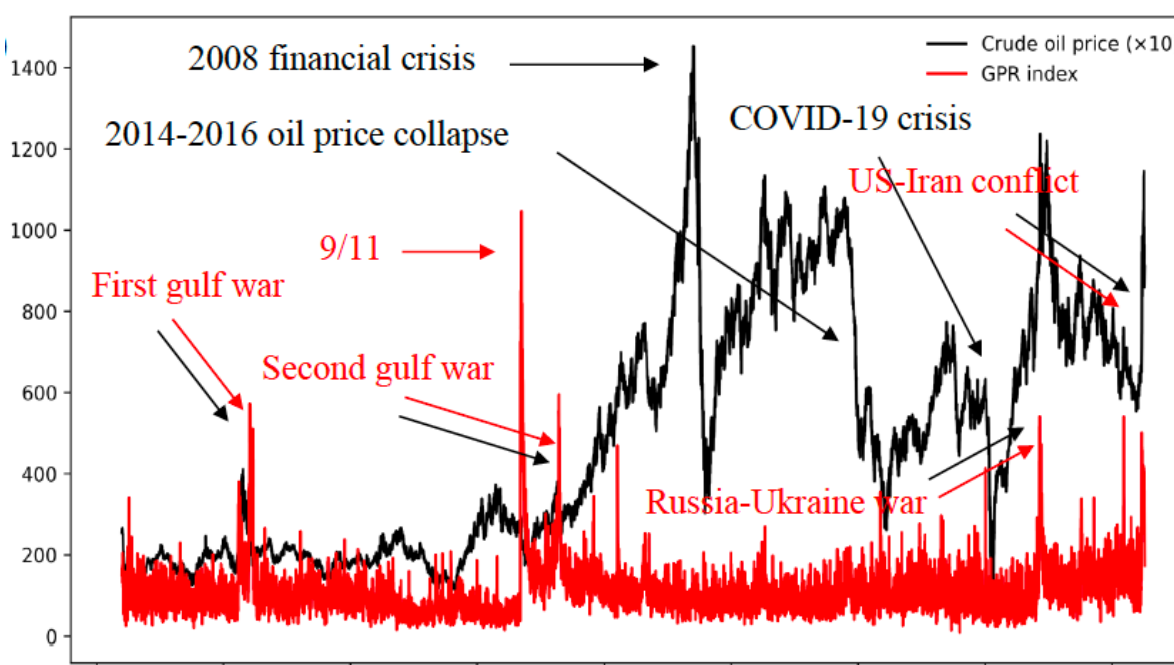
Problem: Discover the dynamic stochastic dependencies of the slow variables when the fast variables are unobserved.

Data: Each sample records $(x_n, \Delta x_{j,n})$, where $\Delta x_{j,n} = x_{j,n+1} - x_{j,n}$.



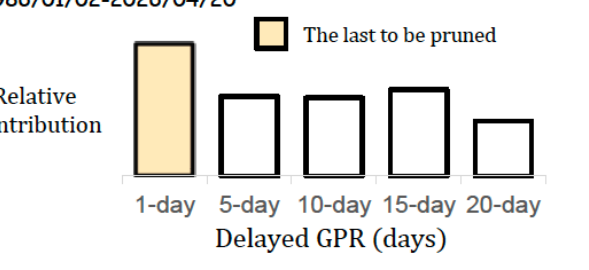
Recovered graph for stochastic transition law $\Delta x_{j,n} \mid x_n$.

Example: Economic Market Data

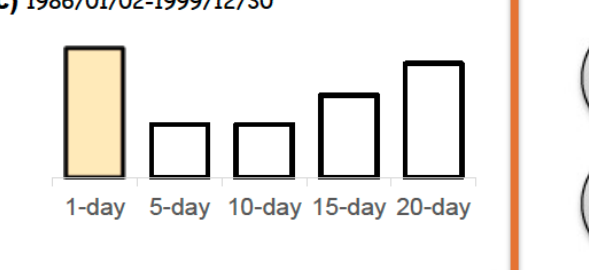


Observed variables: past oil returns and delayed GPR values.

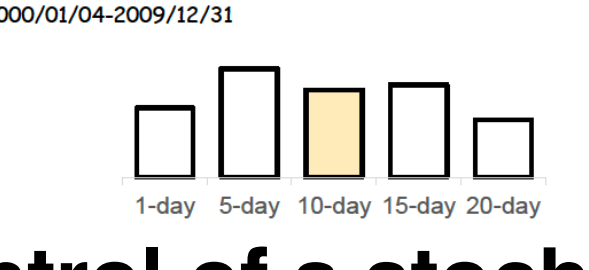
(b) 1986/01/02-2026/04/20



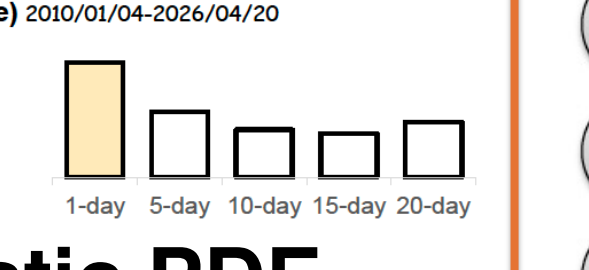
(c) 1986/01/02-1999/12/30



(d) 2000/01/04-2009/12/31



(e) 2010/01/04-2026/04/20

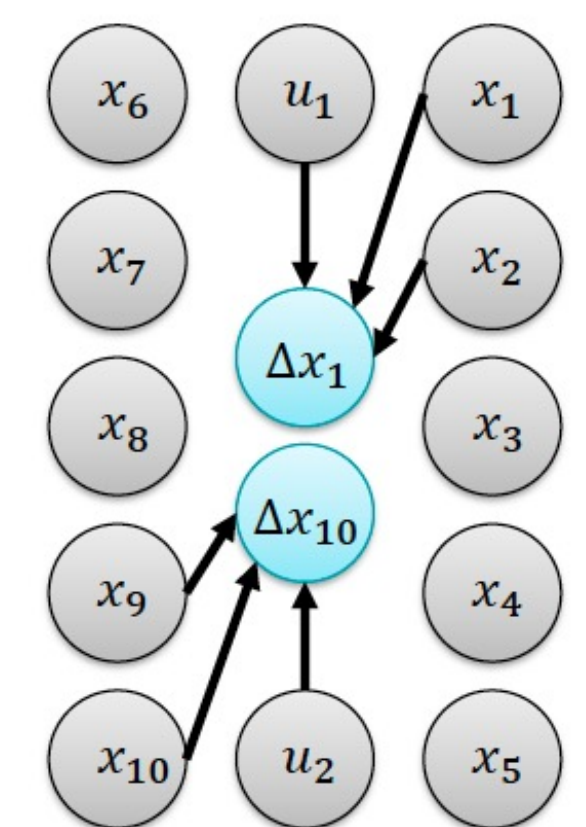


Data-driven boundary control of a stochastic PDE

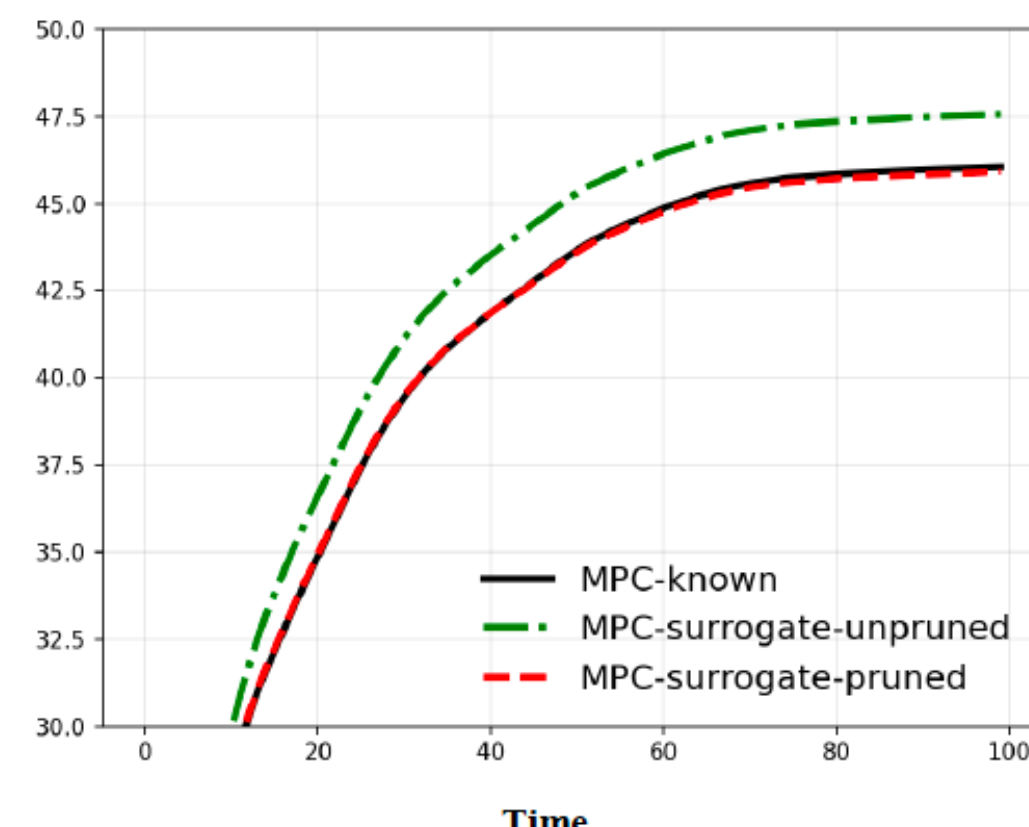
$$\frac{\partial u}{\partial t} = \kappa \frac{\partial^2 u}{\partial x^2} + \sigma u \xi, \quad x \in [0, 1], \quad t > 0, \quad \xi: \text{White noise}$$

Neumann boundary control $-\kappa \frac{\partial u}{\partial x}(0, t) = u_1(t), \quad \kappa \frac{\partial u}{\partial x}(1, t) = u_2(t),$

Recovered graph of stochastic dependencies

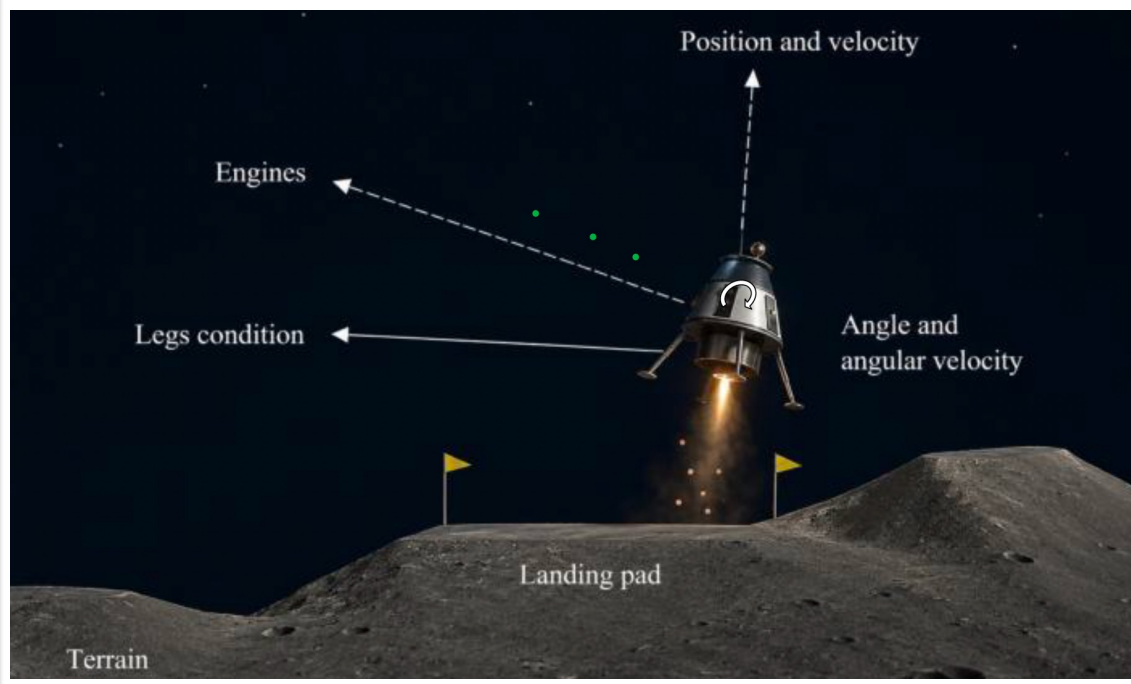


Mean cumulative running cost (state + control)



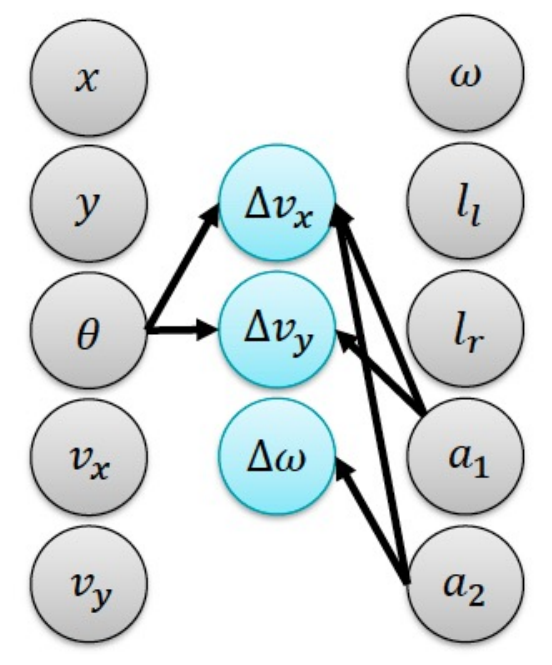
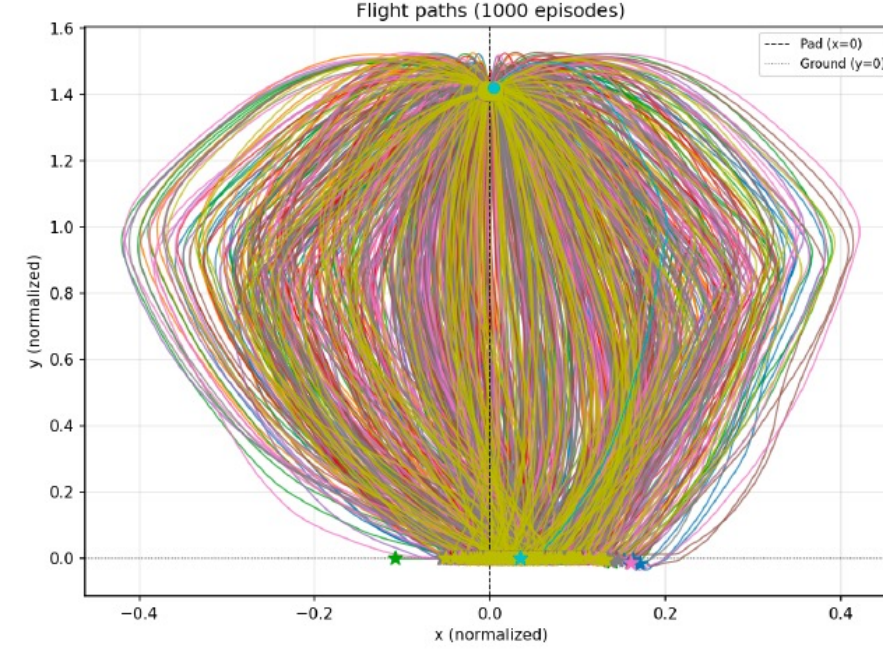
The pruned stochastic twin achieves performance comparable to the known-dynamics MPC, while reducing online MPC computation time.

Lunar Lander



The DT pipeline

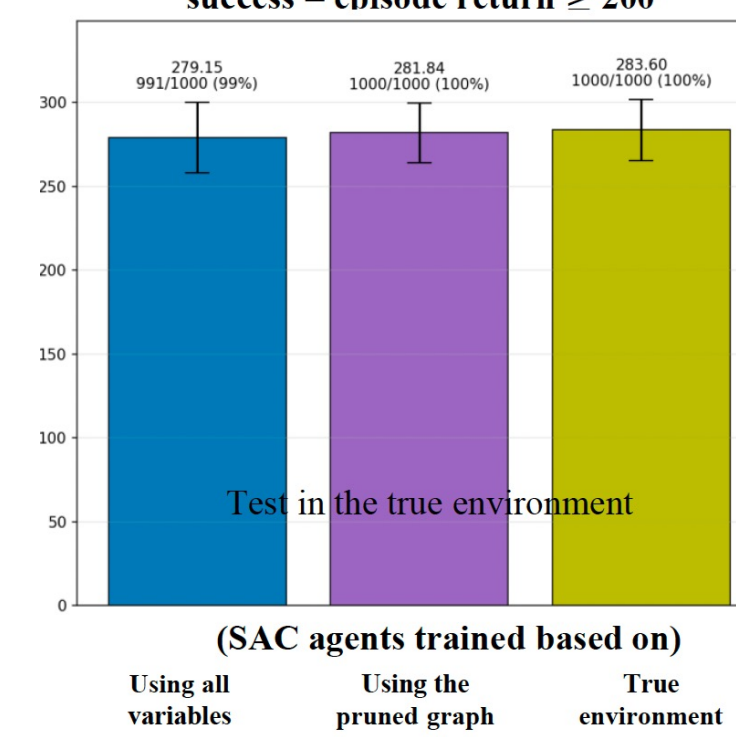
Learn ancestors $\rightarrow$ Learn sparse stochastic digital twin $\rightarrow$ Train SAC controller in the Twin $\rightarrow$ Deploy controller in the true environment

Controlled flight paths from 1000 episodes using the SAC agent trained in the surrogate environment with the pruned ancestor sets.

Soft Actor-Critic (SAC): model-free RL algorithm for continuous control. Learns the state to control map/policy offline then deploy online.

Training the SAC controller in the sparse DT achieves the same return as training in the true environment



Return (sum of rewards) across 1000 episodes
success = episode return ≥ 200

(SAC agents trained based on)

Using all variables: 279.15 (991/1000 (99%))

Using the pruned graph: 281.84 (1000/1000 (100%))

True environment: 283.60 (1000/1000 (100%))

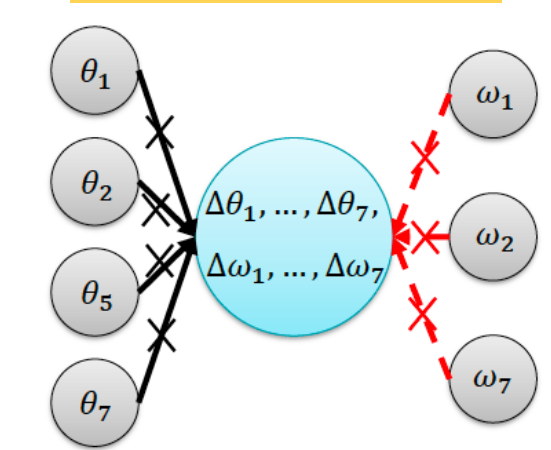
Test in the true environment

7-DOF robotic arm



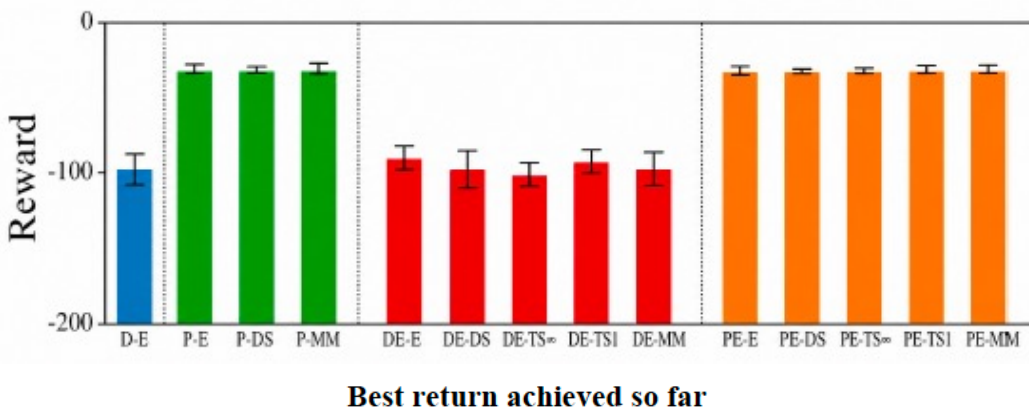
The DT pipeline

Learn ancestors $\rightarrow$ Learn sparse stochastic digital twin $\rightarrow$ Use the twin for online MPC planning (PETS)

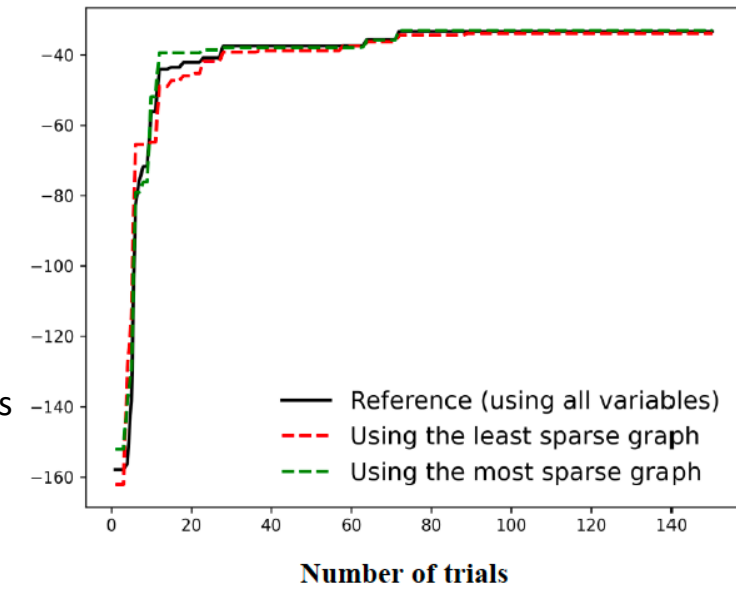


Pruned variables.

MPC: Model Predictive Control
PETS: Probabilistic Ensembles with Trajectory Sampling (online model-based control strategy)



Best return achieved so far



Number of trials

Although the true model is deterministic the learned sparse stochastic model outperforms the learned deterministic model because it incorporates uncertainty due to finite data and it maintains MPC/PETS performance while using fewer state variables